Embedded Module Architecture

ESP32-S3 CAM サンプルプロジェクト

モジュールインターフェース設計思想

バージョン	1.0
作成日	2026-03-11
著者	Shiozawa
ステータス	初版

本ドキュメントは、ESP32-S3 N16R8 CAM 開発ボードを題材に、
組み込みシステム向け Module インターフェースの
設計思想と API を定義する仕様書です。

改訂履歴

版	日付	著者	変更内容
1.0	2026-03-11	Shiozawa	初版作成。IModule、ModuleTimer、SystemData パターンの設計仕様を定義

目次

改訂履歴 1

1 **概要** 3
- 1.1 プロジェクト名 . . . 3
- 1.2 コンセプト . . . 3
- 1.3 設計思想 . . . 3

2 **背景・課題** 3
- 2.1 組み込み開発における一般的な課題 . . . 3
- 2.2 ESP32-S3 CAM の特徴 . . . 4

3 **目的** 4
- 3.1 主目的 . . . 4
- 3.2 副次的目的 . . . 4
- 3.3 範囲 . . . 4

4 **システムアーキテクチャ** 5
- 4.1 全体構造 . . . 5
- 4.2 実行フロー . . . 5
- 4.3 設計パターン . . . 6

5 **ファイル構成** 7
- 5.1 ディレクトリ構造 . . . 7
- 5.2 ファイル一覧と機能対応 . . . 7
- 5.3 ライブラリ構成方針 . . . 8

6 **技術スタック** 8
- 6.1 PlatformIO 設定 . . . 8

7 **API 仕様** 9
- 7.1 IModule インターフェース . . . 9
- 7.2 ModuleTimer クラス . . . 10
- 7.3 SystemData 構造体 . . . 11

8 **実行モデル詳細** 12
- 8.1 ライフサイクル . . . 12
- 8.2 タイミング制御 . . . 12
- 8.3 モジュール間データフロー . . . 12
- 8.4 モジュール実装テンプレート . . . 13

1　概要

1.1　プロジェクト名

Embedded Module Architecture — ESP32-S3 CAM サンプルプロジェクト

1.2　コンセプト

本プロジェクトは、組み込みシステムにおける **Module インターフェース設計パターン**のリファレンス実装である。ESP32-S3 N16R8 CAM 開発ボードを題材に、カメラ・センサー・LED・LCD 等の多様なハードウェアモジュールを統一的なインターフェースで管理する手法を示す。

1.3　設計思想

本アーキテクチャの根幹となる 3 つの要素：

1. **IModule インターフェース** — 全モジュールが `init()` と `update()` を実装する統一的なライフサイクル管理
2. **ModuleTimer クラス** — `delay()` を一切使用しないノンブロッキングなタイマー制御
3. **SystemData パターン** — プロジェクト固有の共有状態をサブ構造体で一元管理し、モジュール間のデータ連携を実現

この設計により、以下を実現する：

- モジュールの追加・削除が容易
- 処理順序が決定論的（ループ内で順番に実行）
- 複数回出力や処理の重複が発生しない
- ゲームループ（ECS）に類似した、組み込みに適した実行モデル

2　背景・課題

2.1　組み込み開発における一般的な課題

組み込みシステムの開発において、以下のような問題が頻繁に発生する：

1. **delay() の乱用**
   ブロッキング関数である `delay()` を使用すると、その間は他の処理が完全に停止する。複数のセンサーやアクチュエータを同時に制御する場合、`delay()` は致命的なボトルネックとなる。
2. **モジュール間の密結合**
   各ハードウェアモジュールの制御コードが `loop()` 内に直接記述されると、コードの肥大化・複雑化が進み、モジュールの追加・削除・変更が困難になる。
3. **出力の重複・不整合**
   統一的なライフサイクル管理がない場合、同一フレーム内で複数回のセンサー読み取りや出力が発生するなど、処理の重複や不整合が起こりやすい。
4. **グローバル変数の氾濫**
   モジュール間のデータ共有にグローバル変数を多用すると、依存関係が不明瞭になり、バグの原因と

なる。

2.2 ESP32-S3 CAM の特徴

本プロジェクトで使用する ESP32-S3 N16R8 CAM 開発ボードは、以下の特徴を持つ：

項目	仕様
MCU	ESP32-S3（デュアルコア Xtensa LX7）
Flash	16MB（N16）
PSRAM	8MB（R8、OPI 接続）
通信	WiFi + Bluetooth 5.0
カメラ	OV2640 / OV5640 対応
フレームワーク	Arduino

表 1　ESP32-S3 N16R8 CAM 主要スペック

多機能なハードウェアを効率的に管理するために、統一的なモジュール管理アーキテクチャが求められる。

3 目的

本プロジェクトの目的は以下の通りである：

3.1 主目的

1. **Module インターフェース設計パターンの確立**
   組み込みシステムにおけるハードウェアモジュール管理の設計パターンを定義し、リファレンス実装として公開する。
2. **再利用可能なアーキテクチャの提供**
   ESP32-S3 に限らず、他のマイコンプラットフォームでも適用可能な汎用的な設計として、`IModule` インターフェース、`ModuleTimer` クラス、`SystemData` パターンを提供する。

3.2 副次的目的

1. **教育・学習用のサンプル**
   組み込み開発の初学者が、モジュール化・非ブロッキング処理・データフロー管理を学べる具体的なコード例を提供する。
2. **ESP32-S3 CAM の活用例**
   カメラ、センサー、LED、LCD 等の周辺機器を組み合わせた実用的なサンプルアプリケーションを構築する。

3.3 範囲

本プロジェクトが提供するもの：

- IModule インターフェースの定義と仕様

- ModuleTimer クラスの実装
- SystemData パターンの設計指針
- ESP32-S3 CAM を用いたサンプルモジュール群

4　システムアーキテクチャ

4.1　全体構造

本アーキテクチャは、以下の 3 つのコア要素で構成される：

1. **IModule**（インターフェース） — モジュールのライフサイクルを定義
2. **ModuleTimer**（ユーティリティクラス） — ノンブロッキングなタイマー制御
3. **SystemData**（共有データ構造体） — モジュール間のデータ連携

4.2　実行フロー

プログラムの実行フローは、ゲームループパターンに基づく：

Listing 1　基本的な実行フロー

```cpp
// main.cpp
#include "IModule.h"
#include "SystemData.h"
#include "CameraModule.h"
#include "LedModule.h"

// システムデータ
SystemData systemData;

// モジュールインスタンス
CameraModule cameraModule;
LedModule ledModule;

// モジュール配列（処理順序を定義）
IModule* modules[] = {
    &cameraModule,
    &ledModule,
};
const int MODULE_COUNT = sizeof(modules) / sizeof(modules[0]);

void setup() {
    Serial.begin(115200);

    // 全モジュールを初期化
    for (int i = 0; i < MODULE_COUNT; i++) {
        if (!modules[i]->init()) {
            Serial.printf("Module %d: init failed\n", i);
        }
    }
}

void loop() {
```

```
    // 全モジュールを順番に更新
    for (int i = 0; i < MODULE_COUNT; i++) {
        modules[i]->update(&systemData);
    }
}
```

4.3 設計パターン

4.3.1 モジュール配列パターン

`modules[]` 配列の順序がそのまま処理の実行順序となる。これにより：

- 処理順序が明示的かつ決定論的
- データの流れが配列の順序に沿って一方向
- 1 ループにつき各モジュール 1 回の更新が保証される

重要：処理順序

`modules[]` 配列の順序は、データフローの順序を意味する。
例：カメラモジュールが画像を取得 → LCD モジュールがその画像を表示
この場合、カメラモジュールを LCD モジュールより前に配置する必要がある。

4.3.2 SystemData パターン

各モジュールは固有のサブ構造体を定義し、それらをプロジェクト固有の `SystemData` 構造体に集約する。

Listing 2　SystemData パターンの構造

```
// 各モジュールが自身のデータ構造体を定義
// LedModule.h 内
struct LedData {
    bool state;
    uint8_t brightness;
};

// TempSensorModule.h 内
struct TempData {
    float temperature;
    bool isValid;
    bool hasError;
};

// プロジェクト固有: SystemData.h
struct SystemData {
    LedData led;
    TempData temp;
    // 使用するモジュールの構造体を追加するだけ
};
```

4.3.3 エラーハンドリングパターン

- `init()` — 戻り値 `bool` で初期化の成否を返す。失敗時はモジュールを無効化可能

- update() — エラー発生時は SystemData 内のサブ構造体にエラーフラグを設定し、他モジュールから参照可能にする

5 ファイル構成

5.1 ディレクトリ構造

```
Embedded-Module-Architecture/
+-- .devcontainer/
|   +-- Dockerfile              ... 環境用コンテナ定義LaTeX
|   +-- devcontainer.json       ... 設定DevContainer
+-- doc/
|   +-- .latexmkrc              ... 設定latexmk
|   +-- main.tex                ... 仕様書本体
|   +-- sections/               ... 仕様書の各セクション
|       +-- cover.tex
|       +-- revision.tex
|       +-- overview.tex
|       +-- background.tex
|       +-- purpose.tex
|       +-- architecture.tex
|       +-- files.tex
|       +-- techstack.tex
|       +-- api.tex
|       +-- execution.tex
+-- include/                    ... プロジェクト固有ヘッダー
|   +-- SystemData.h            ... 構造体定義SystemData
+-- lib/
|   +-- ModuleCore/             ... コアライブラリ
|       +-- IModule.h           ... インターフェースIModule
|       +-- ModuleTimer.h       ... クラスModuleTimer
+-- src/
|   +-- main.cpp                ... エントリーポイント
+-- test/                       ... テストコード
+-- platformio.ini              ... 設定PlatformIO
+-- README.md                   ... プロジェクト概要
```

5.2 ファイル一覧と機能対応

ファイル	種別	説明
lib/ModuleCore/IModule.h	コア	IModule インターフェース定義
lib/ModuleCore/ModuleTimer.h	コア	ModuleTimer クラス定義
include/SystemData.h	プロジェクト固有	SystemData 構造体（使用するモジュールのサブ構造体を集約）
src/main.cpp	プロジェクト固有	エントリーポイント（モジュール登録・ループ制御）

5.3 ライブラリ構成方針

- **lib/ModuleCore/** — プロジェクト非依存のコアコンポーネント（IModule、ModuleTimer）。他のプロジェクトにもそのまま流用可能。
- **lib/各モジュール名/** — 個別のハードウェアモジュール実装。それぞれが `IModule` を継承し、モジュール固有のサブ構造体を定義する。
- **include/** — プロジェクト固有のヘッダー（`SystemData.h` 等、プロジェクトで使用するモジュールの組み合わせに依存するファイル）。

6 技術スタック

カテゴリ	技術	備考
MCU	ESP32-S3 N16R8	デュアルコア Xtensa LX7、16MB Flash、8MB PSRAM
カメラ	OV2640 / OV5640	開発ボード付属
フレームワーク	Arduino	PlatformIO 経由
ビルドツール	PlatformIO	VSCode 統合開発環境
言語	C++	Arduino C++（C++11 以降）
仕様書	LaTeX (uplatex)	DevContainer + LaTeX Workshop
バージョン管理	Git	—

6.1 PlatformIO 設定

Listing 3 platformio.ini

```
[env:esp32-s3-cam-n16r8]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200

; --- N16 (16MB Flash) ---
board_upload.flash_size = 16MB
board_build.partitions = default_16MB.csv

; --- R8 (8MB PSRAM) ---
board_build.arduino.memory_type = qio_opi
build_flags =
    -D BOARD_HAS_PSRAM
    -D CORE_DEBUG_LEVEL=3
```

7　API 仕様

7.1　IModule インターフェース

IModule — モジュールインターフェース

定義ファイル: `lib/ModuleCore/IModule.h`

全ハードウェアモジュールが実装すべき抽象インターフェース。`init()` と `update()` の 2 つの仮想関数により、統一的なライフサイクル管理を実現する。

Listing 4　IModule インターフェース定義

```
class IModule {
public:
    // モジュールの初期化
    // 戻り値: true成功=, false失敗=
    virtual bool init() = 0;

    // モジュールの更新（毎ループ呼び出される）
    // 引数: sys - プロジェクト共有データへのポインタ
    virtual void update(SystemData* sys) = 0;

    // 仮想デストラクタ
    virtual ~IModule() {}
};
```

7.1.1　init()

項目	内容
戻り値	`bool` — `true` で初期化成功、`false` で失敗
呼び出しタイミング	`setup()` 内で 1 回のみ
責務	ハードウェアの初期化、ピン設定、ModuleTimer の `setTime()` 呼び出し等
エラー時	`false` を返す。呼び出し側でモジュールの無効化やログ出力を行う

7.1.2　update(SystemData* sys)

項目	内容
引数	`SystemData*` — プロジェクト共有データへのポインタ
戻り値	`void`
呼び出しタイミング	`loop()` 内で毎ループ呼び出される
責務	センサー読み取り、出力制御、SystemData の更新
エラー時	SystemData 内のサブ構造体にエラーフラグを設定

7.2 ModuleTimer クラス

ModuleTimer — ノンブロッキングタイマー

定義ファイル: `lib/ModuleCore/ModuleTimer.h`

`millis()` ベースのシンプルなタイマークラス。`delay()` を使用せずに周期実行・デバウンス・ワンショット・タイムアウト等を実現する。インスタンスを複数作成することで、モジュール内で複数のタイマーを独立に管理できる。

Listing 5　ModuleTimer クラス定義

```cpp
class ModuleTimer {
private:
    unsigned long _startTime;

public:
    // コンストラクタ
    ModuleTimer() : _startTime(0) {}

    // 基準時刻をセット
    // 引数なし: getNowTime()から開始0
    // 引数あり: getNowTime()が指定値から開始
    void setTime(unsigned long offsetMs = 0) {
        _startTime = millis() - offsetMs;
    }

    // 基準時刻からの経過時間(ms)を取得
    unsigned long getNowTime() const {
        return millis() - _startTime;
    }
};
```

7.2.1 setTime(unsigned long offsetMs = 0)

項目	内容
引数	`offsetMs`（デフォルト: 0） — 初期オフセット値 (ms)
戻り値	`void`
動作	内部の基準時刻を `millis() - offsetMs` に設定する。`offsetMs = 0` の場合、直後の `getNowTime()` は 0 を返す。`offsetMs = 100` の場合、直後の `getNowTime()` は 100 を返す
用途	タイマーの開始・リセット。`init()` 内および周期リセット時に使用

7.2.2 getNowTime() const

項目	内容
引数	なし
戻り値	`unsigned long` — 最後の `setTime()` からの経過時間 (ms)
動作	`millis() - _startTime` を返す
用途	経過時間の判定。条件分岐と組み合わせて周期実行・デバウンス等を実現

7.2.3 使用パターン

Listing 6　周期実行パターン

```
ModuleTimer readTimer;

void init() {
    readTimer.setTime();
}

void update(SystemData* sys) {
    if (readTimer.getNowTime() >= 1000) {
        readTimer.setTime();  // リセット
        sys->temp.temperature = readSensor();
    }
}
```

Listing 7　デバウンスパターン

```
ModuleTimer debounce;

void update(SystemData* sys) {
    bool rawInput = digitalRead(BUTTON_PIN);

    if (rawInput != lastRawInput) {
        debounce.setTime();  // 入力変化でリセット
        lastRawInput = rawInput;
    }

    if (debounce.getNowTime() >= 50) {
        sys->button.pressed = lastRawInput;  // 50安定で確定ms
    }
}
```

7.3　SystemData 構造体

SystemData — プロジェクト共有データ

定義ファイル: `include/SystemData.h`（プロジェクト固有）
各モジュールが定義するサブ構造体を集約した、プロジェクト固有のデータ構造体。全モジュールの `update()` メソッドに共有ポインタとして渡される。

7.3.1　設計ルール

1. 各モジュールは、自身が管理するデータをサブ構造体として定義する（モジュールのヘッダーファイル内）
2. `SystemData` 構造体は、プロジェクトで使用するモジュールのサブ構造体をメンバとして持つ
3. モジュールは `update()` 内で、自身のサブ構造体に書き込み、必要に応じて他モジュールのサブ構造体を読み取る

Listing 8　SystemData 構成例

```
// SystemData.h
#pragma once
```

```
#include "LedModule.h"
#include "TempSensorModule.h"
#include "CameraModule.h"

struct SystemData {
    LedData led;
    TempData temp;
    CameraData camera;
};
```

8 実行モデル詳細

8.1 ライフサイクル

本アーキテクチャのモジュールは、以下のライフサイクルに従う：

1. **インスタンス生成**（グローバルスコープ）
   モジュールのコンストラクタが呼ばれる。ModuleTimer のメンバ変数が`_startTime = 0`で初期化される。この時点では`millis()`は信頼できないため、タイマーの実質的な初期化は行わない。
2. **init() 呼び出し**（`setup()`内）
   ハードウェアの初期化、ピンモード設定、ModuleTimer の`setTime()`呼び出しを行う。失敗時は`false`を返し、呼び出し側でエラーハンドリングを行う。
3. **update() ループ**（`loop()`内）
   毎ループ呼び出される。ModuleTimer の`getNowTime()`で経過時間を確認し、必要なタイミングでのみ実処理を実行する。

8.2 タイミング制御

`delay()`は一切使用しない。すべてのタイミング制御は ModuleTimer で行う。

パターン	方法	コード例
周期実行	`getNowTime() >= interval`で判定し、`setTime()`でリセット	1 秒ごとのセンサー読み取り
デバウンス	入力変化時に`setTime()`でリセット、安定後に確定	ボタンのチャタリング除去
ワンショット	`getNowTime()`で経過時間を確認、フラグで制御	起動後 N 秒で LED 消灯
タイムアウト	`getNowTime() >= timeout`で時間超過を検出	WiFi 接続のタイムアウト

8.3 モジュール間データフロー

`SystemData`を介したデータフローの例：

Listing 9　モジュール間データフローの例

```
// 温度センサーモジュール（先に実行）
void TempSensorModule::update(SystemData* sys) {
    if (readTimer.getNowTime() >= 1000) {
        readTimer.setTime();
```

```
        sys->temp.temperature = readSensor();
        sys->temp.isValid = true;
    }
}

// モジュール（後に実行）LED
void LedModule::update(SystemData* sys) {
    // 温度データを読み取って、温度に応じてを制御LED
    if (sys->temp.isValid && sys->temp.temperature > 30.0) {
        sys->led.state = true;   // 度超で30点灯LED
    } else {
        sys->led.state = false;
    }
    digitalWrite(LED_PIN, sys->led.state);
}
```

8.4 モジュール実装テンプレート

新規モジュールを追加する際の標準テンプレート：

Listing 10 モジュール実装テンプレート

```
// ExampleModule.h
#pragma once
#include "IModule.h"
#include "ModuleTimer.h"

// このモジュール固有のデータ構造体
struct ExampleData {
    // モジュールが管理するデータを定義
    int value;
    bool hasError;
};

class ExampleModule : public IModule {
private:
    ModuleTimer _updateTimer;  // メンバ変数としてタイマー保持

public:
    ExampleModule() {}  // コンストラクタ

    bool init() override {
        _updateTimer.setTime();  // タイマー初期化
        // ハードウェア初期化処理...
        return true;
    }

    void update(SystemData* sys) override {
        if (_updateTimer.getNowTime() >= 100) {
            _updateTimer.setTime();
            // 更新処理...
        }
    }
};
```